

Graph-Based Imitation and Reinforcement Learning for Efficient Benders Decomposition

Bernard T. Agyeman^a, Zhe Li^a, Ilias Mitrai^{b*}, Prodromos Daoutidis^{a†}

^aDepartment of Chemical Engineering and Material Science, University of Minnesota, Minneapolis, MN 55455, United States.

^bMcKetta Department of Chemical Engineering, University of Texas at Austin, Austin, TX 78712, United States.

Abstract

This work introduces an end-to-end graph-based agent for accelerating the computational efficiency of Benders Decomposition. The agent’s policy is parameterized by a graph neural network which takes as input a bipartite graph representation of the master problem and proposes a candidate solution. The agent is trained using a two-stage approach that combines imitation (IL) and reinforcement learning (RL). IL is used to mimic a solver and obtain a warm-start policy which is then finetuned using RL with a reward signal that balances feasibility and computational efficiency. We augment the agent with a verification mechanism that checks the agent’s prediction for feasibility and solution quality. The framework is evaluated in two case studies: (i) an illustrative mixed-integer nonlinear program, where it reduces the solution time by 42% without loss of solution quality, and (ii) a closed-loop irrigation scheduling problem, where it achieves a 23% time reduction without compromising water use efficiency.

Keywords: Reinforcement learning, Imitation learning, Benders decomposition, Mixed-integer programming, Graph neural networks.

*Corresponding author: Ilias Mitrai. Email: imitrai@che.utexas.edu.

†Corresponding author: Prodromos Daoutidis. Email: daout001@umn.edu.

1 Introduction

Large-scale mixed-integer optimization problems arise in many chemical engineering applications, including process design, operation, and capacity expansion [1], as well as parameter estimation [2]. A common feature of such problems is the presence of complicating variables, which, if fixed, render the resulting problem easier to solve. This structure is the basis of Benders decomposition (BD), where the original problem is decomposed into a master mixed-integer problem and a continuous subproblem [3]. The master problem and subproblem are solved sequentially and coordinated via duality, namely, Benders optimality and feasibility cuts. These cuts inform the master problem about the effect of the complicating variables on the subproblem. Originally developed for mixed-integer linear programming (MILP) problems, BD was later extended to convex mixed-integer nonlinear programming (MINLP) problems, giving rise to generalized Benders decomposition (GBD) [4].

Despite the wide use of BD in domains such as supply chains [5], power systems [6], and mixed integer model predictive control (MPC) [7], its off-the-shelf implementation can be challenging. In particular, its computational performance depends on the complexity of the master problem and subproblem as well as the quality of the cuts generated. Several acceleration techniques have been proposed to improve performance [8]. At a high level, these techniques exploit tailored problem decompositions (e.g., partial decomposition [9] and multicut implementations [10]), approximate solution of the subproblem [11, 7], and advanced cut generation techniques (e.g., Pareto-optimal cuts [12]).

In this paper, we focus on the master problem whose complexity increases as Benders cuts are added during the solution process. Usually, the master problem is solved to global optimality at each iteration, which can be computationally expensive. This has motivated the use of heuristics, such as variable neighborhood search [13], genetic algorithms [14], and tabu search [15], to quickly obtain feasible solutions. However, these methods generally do not provide optimality guarantees [16]. An alternative is to solve the master problem to local optimality in early iterations, when the relaxation is weak [17], and recent work builds on this idea by using reinforcement learning (RL) to control the degree of inexactness across iterations [18]. Although locally optimal solutions of the master problem can still guarantee convergence to the globally optimal solution when the subproblem is convex, they may still require substantial computational effort. Given these challenges and the iterative nature of BD, we propose a machine-learning approach, specifically the application of RL

and imitation learning (IL), to train an agent to solve the master problem at each iteration.

Machine learning (ML) has been widely used to accelerate the solution of optimization problems by accelerating the solution process itself [19]. For example, ML has been used to learn branching policies [20], select cutting-planes [21]), estimate Benders cuts [22] and which subproblems to solve [23], predict good initial guess [24], tuning general purpose solvers [25], and select the best solution strategy [26]. Despite these successes, the direct application of RL and IL to the master problem poses three challenges. First, the interaction between decision variables and constraints in the master problem changes across iterations as Benders cuts are added. Classical vectorial feature representations of optimization problems, which are used as inputs to ML models, cannot capture this evolving interaction pattern explicitly [27]. Second, applying RL in combinatorial settings is challenging due to the large decision space [28]. Finally, predictions from ML-based optimization approaches do not, in general, provide feasibility or optimality guarantees, which can compromise key aspects of the BD algorithm if incorporated naively.

To address the representation challenge, we encode each master problem instance using a bipartite graph. Graphs have been used extensively to capture structural interactions between variables and constraints in optimization problems [29]. The graph is augmented with node and edge features that encode information such as variable domains, right-hand side (RHS) values of constraints, and coefficients of variables in constraints. In the context of BD, these features capture information relevant to optimality and feasibility cuts. We then parameterize the agent’s policy with a graph neural network (GNN), which operates on graph-structured inputs. To mitigate the learning difficulties in the combinatorial setting of the master problem, we adopt a two-step training procedure: an initial policy is learned from expert demonstrations via IL, and the policy is subsequently fine-tuned using RL with a reward that captures key aspects of the BD algorithm. Finally, we introduce a verification and coordination mechanism to enable a robust integration of the learned policy into BD, ensuring that the lack of feasibility and optimality guarantees does not compromise key aspects of the algorithm.

Graph-based RL has been studied for a range of combinatorial optimization problems; see, for example, the survey by [30]. The contribution in this work is not the general use of graph-based RL for combinatorial optimization, but its use to accelerate the BD master problem and, consequently, the overall BD algorithm. This setting differs from many existing graph-based RL formulations in that (i) the structure of the master problem changes across iterations as Benders cuts accumulate, and (ii) the agent’s proposed actions must be evaluated in a BD-specific context,

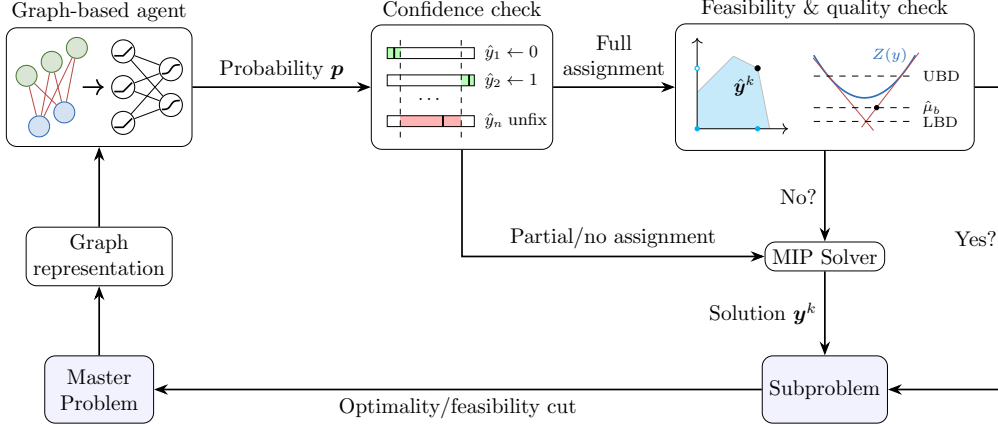


Figure 1: Diagram of the proposed framework.

including feasibility with respect to the current master problem, progress in reducing the bound gap, and the induced computational effort required to solve the subproblem.

Overall, this paper proposes a graph-based IL and RL agent and verifier for reducing the computational effort related to the BD (see Figure 1). Specifically, the contributions of this paper are:

1. **Graph-based agent for the master problem:** A graph-based agent is developed to approximately solve the master problem. The agent’s policy is parameterized by a GNN with a multi-headed output layer, which takes as input a bipartite graph representation of the master problem, and outputs a candidate solution. The agent’s design combines IL and RL in a complementary way: IL is first used to obtain a warm-start policy by imitating a mixed-integer programming (MIP) solver, and this policy initializes the actor of an actor-critic RL framework. RL then fine-tunes the policy using a hybrid reward that balances (i) feasibility with respect to the current master problem, (ii) progress in reducing the gap between the upper and lower bounds, and (iii) the computational effort required to solve the subproblem.
2. **Verification and coordination mechanism:** A mechanism, called a confidence-based assignment, is developed to coordinate the hybrid use of the agent’s policy and an MIP solver within BD when solving the master problem. This mechanism ensures that only feasible and sufficiently good solutions are forwarded to the subproblem. It combines prediction confidence, feasibility, and cost consistency checks to determine when to use the agent’s actions and when to defer to the solver.

The proposed framework is evaluated on two optimization problems: (i) an illustrative MINLP,

and (ii) a closed-loop irrigation scheduling problem modeled as a mixed-integer MPC. The remainder of the paper is organized as follows. Section 2 introduces the preliminaries, including the class of problems considered, the classical GBD algorithm, graph-based representations optimization problems, and an overview of GNNs, RL and IL. Section 3 presents the graph-based agent, covering the bipartite graph of the master problem, the IL stage, and the RL stage. Section 4 presents the adapted confidence-based approach. Section 5 reports and discusses the results of two case studies, and Section 6 concludes the paper.

2 Preliminaries

2.1 Problem Formulation

Without loss of generality, this study focuses on GBD. We consider the solution of MINLP problems in the following general form:

$$\begin{aligned}
& \min_{\mathbf{x}, \mathbf{y}} F(\mathbf{x}, \mathbf{y}) \\
& \text{s.t. } \mathbf{H}(\mathbf{x}, \mathbf{y}) = \mathbf{0}, \\
& \quad \mathbf{G}(\mathbf{x}, \mathbf{y}) \leq \mathbf{0}, \\
& \quad \mathbf{K}\mathbf{y} - \mathbf{b} \leq \mathbf{0}, \\
& \quad \mathbf{x} \in \mathcal{X}, \quad \mathbf{y} \in \{0, 1\}^m,
\end{aligned} \tag{1}$$

where $\mathbf{x} \in \mathbb{R}^n$ are continuous variables, $\mathbf{y} \in \{0, 1\}^m$ are binary variables, $\mathbf{K} \in \mathbb{R}^{o \times m}$, and $\mathbf{b} \in \mathbb{R}^o$. Vector-valued equality and inequality constraints are given by $\mathbf{H} : \mathbb{R}^n \times \{0, 1\}^m \rightarrow \mathbb{R}^p$ and $\mathbf{G} : \mathbb{R}^n \times \{0, 1\}^m \rightarrow \mathbb{R}^q$, respectively. The set $\mathcal{X}$ is defined as $\mathcal{X} = \{\mathbf{x} \in \mathbb{R}^n \mid \mathbf{E}\mathbf{x} \leq \mathbf{d}, \underline{\mathbf{x}} \leq \mathbf{x} \leq \bar{\mathbf{x}}\}$, where $\mathbf{E} \in \mathbb{R}^{l \times n}$, $\mathbf{d} \in \mathbb{R}^l$, and $\underline{\mathbf{x}}$, $\bar{\mathbf{x}}$ denote the lower and upper bounds on $\mathbf{x}$, respectively. We focus specifically on a subclass of formulation (1) in which $\mathbf{y}$ enters the problem linearly in both the objective function and the constraints, while $\mathbf{x}$ appears nonlinearly. Specifically, we assume:

$$F(\mathbf{x}, \mathbf{y}) := f(\mathbf{x}) + \mathbf{e}^\top \mathbf{y}, \quad \mathbf{H}(\mathbf{x}, \mathbf{y}) := \mathbf{h}(\mathbf{x}) + \mathbf{A}\mathbf{y}, \quad \mathbf{G}(\mathbf{x}, \mathbf{y}) := \mathbf{g}(\mathbf{x}) + \mathbf{B}\mathbf{y},$$

where $\mathbf{e} \in \mathbb{R}^m$, $\mathbf{A} \in \mathbb{R}^{p \times m}$ and $\mathbf{B} \in \mathbb{R}^{q \times m}$ are given. The scalar function $f : \mathbb{R}^n \rightarrow \mathbb{R}$, and the vector-valued function $\mathbf{g} : \mathbb{R}^n \rightarrow \mathbb{R}^q$, are assumed to be continuous, differentiable, and convex. In addition, the vector-valued function $\mathbf{h} : \mathbb{R}^n \rightarrow \mathbb{R}^p$ is assumed to be linear in $\mathbf{x}$.

2.2 Generalized Benders Decomposition (GBD)

GBD [4] proceeds by iteratively refining upper and lower bounds on the optimal solution of the original MINLP problem. In this work, we consider equality constraints and follow the GBD algorithm described in [31]. For formulation (1), the upper bound (UBD) is obtained from the subproblem, while the lower bound (LBD) is provided by the master problem. In iteration k , the subproblem corresponds to the original MINLP with fixed binary variables $\mathbf{y}^k$. This problem, denoted as $\mathcal{S}(\mathbf{y}^k)$, is defined as:

$$\begin{aligned} Z(\mathbf{y}^k) &= \min_{\mathbf{x}} F(\mathbf{x}, \mathbf{y}^k) \\ \text{s.t. } \quad &\mathbf{H}(\mathbf{x}, \mathbf{y}^k) = \mathbf{0}, \\ &\mathbf{G}(\mathbf{x}, \mathbf{y}^k) \leq \mathbf{0}, \\ &\mathbf{x} \in \mathcal{X}. \end{aligned}$$

If the subproblem $\mathcal{S}(\mathbf{y}^k)$ is feasible, its objective value provides an UBD to the original MINLP. Otherwise, a feasibility subproblem is solved to generate a feasibility cut. This subproblem, denoted as $\mathcal{F}(\mathbf{y}^k)$, is formulated by introducing slack variables $\boldsymbol{\alpha}$ as:

$$\begin{aligned} \min_{\mathbf{x}, \boldsymbol{\alpha}} \quad &\sum_{i=1}^q \alpha_i \\ \text{s.t. } \quad &\mathbf{H}(\mathbf{x}, \mathbf{y}^k) = \mathbf{0}, \\ &G_i(\mathbf{x}, \mathbf{y}^k) \leq \alpha_i, \quad i = 1, \dots, q, \\ &\alpha_i \geq 0, \quad i = 1, \dots, q, \\ &\mathbf{x} \in \mathcal{X}. \end{aligned}$$

Under this decomposition, the original MINLP can be reformulated as

$$\begin{aligned} \min_{\mathbf{y}} \quad &Z(\mathbf{y}) \\ \text{s.t. } \quad &\mathbf{K}\mathbf{y} - \mathbf{b} \leq \mathbf{0}, \\ &\mathbf{y} \in \{0, 1\}^m. \end{aligned} \tag{2}$$

Although this is an exact reformulation, the value function of the subproblem $Z(\mathbf{y})$ is not known explicitly. In Benders decomposition, the value function $Z(\mathbf{y})$ is approximated via hyperplanes, also known as Benders cuts. $\mathcal{C}_o^{(k)}$ is given by: The Benders optimality cut $\mathcal{C}_o^{(k)}$ is given by:

$$\mu_b \geq F(\mathbf{x}^k, \mathbf{y}) + \boldsymbol{\lambda}_k^\top \mathbf{H}(\mathbf{x}^k, \mathbf{y}) + \boldsymbol{\mu}_k^\top \mathbf{G}(\mathbf{x}^k, \mathbf{y}), \tag{3}$$

where $\mathbf{x}^k$ is an optimal solution to $\mathcal{S}(\mathbf{y}^k)$, and $\boldsymbol{\lambda}_k$ and $\boldsymbol{\mu}_k$ are the corresponding dual multipliers of the equality constraints $\mathbf{H}(\mathbf{x}, \mathbf{y}^k) = \mathbf{0}$ and the inequality constraints $\mathbf{G}(\mathbf{x}, \mathbf{y}^k) \leq \mathbf{0}$, respectively. We compactly represent the RHS of the optimality cut $\mathcal{C}_o^{(k)}$ in inequality (3) by $\mathcal{O}_k(\mathbf{y}) := F(\mathbf{x}^k, \mathbf{y}) + \boldsymbol{\lambda}_k^\top \mathbf{H}(\mathbf{x}^k, \mathbf{y}) + \boldsymbol{\mu}_k^\top \mathbf{G}(\mathbf{x}^k, \mathbf{y})$. If the subproblem is infeasible, then the following feasibility cut $\mathcal{C}_f^{(k)}$ is added:

$$\bar{\boldsymbol{\lambda}}_k^\top \mathbf{H}(\bar{\mathbf{x}}^k, \mathbf{y}) + \bar{\boldsymbol{\mu}}_k^\top \mathbf{G}(\bar{\mathbf{x}}^k, \mathbf{y}) \leq 0, \quad (4)$$

where $\bar{\mathbf{x}}^k$ is an optimal solution to $\mathcal{F}(\mathbf{y}^k)$, and $\bar{\boldsymbol{\lambda}}_k$ and $\bar{\boldsymbol{\mu}}_k$ are the corresponding dual multipliers associated with the equality constraints $\mathbf{H}(\mathbf{x}, \mathbf{y}^k) = \mathbf{0}$ and the relaxed inequality constraints $\mathbf{G}(\mathbf{x}, \mathbf{y}^k) \leq \boldsymbol{\alpha}$, respectively. Overall, the master problem, denoted as $\mathcal{M}$, is equal to

$$\begin{aligned} & \min_{\mu_b, \mathbf{y}} \mu_b \\ \text{s.t. } & \mu_b \geq F(\mathbf{x}^k, \mathbf{y}) + \boldsymbol{\lambda}_k^\top \mathbf{H}(\mathbf{x}^k, \mathbf{y}) + \boldsymbol{\mu}_k^\top \mathbf{G}(\mathbf{x}^k, \mathbf{y}), \quad \forall k \in K_O \\ & \bar{\boldsymbol{\lambda}}_k^\top \mathbf{H}(\bar{\mathbf{x}}^k, \mathbf{y}) + \bar{\boldsymbol{\mu}}_k^\top \mathbf{G}(\bar{\mathbf{x}}^k, \mathbf{y}) \leq 0, \quad \forall k \in K_F \\ & \mathbf{K}\mathbf{y} - \mathbf{b} \leq \mathbf{0}, \\ & \mathbf{y} \in \{0, 1\}^m \end{aligned}$$

where K_O and K_F denote the sets of iterations where the subproblem is feasible and infeasible, respectively. In addition to the optimality and feasibility cuts, the master problem includes the original MINLP constraints involving only the binary variables. We refer to these as pure-binary constraints and denote them by $\mathcal{C}_p$:

$$\mathbf{K}\mathbf{y} - \mathbf{b} \leq \mathbf{0}. \quad (5)$$

Since the number of Benders feasibility and optimality cuts can grow large, they are added adaptively during the solution process. Thus, at each iteration, solving the current master problem provides a lower bound (LBD). The complete GBD algorithm (see Algorithm 1 in the Supplementary Material) alternates between solving the master problem and the subproblem until the gap between UBD and LBD falls below a specified tolerance ε .

2.3 Graph Representation of Optimization Problems

Graphs $\mathcal{G}$ are mathematical structures consisting of nodes $\mathcal{V}$ and edges $\mathcal{E}$. Bipartite graphs are especially useful for representing optimization problems, and they have been used in mixed-integer optimization [32] as well as in community-detection approaches that guide decomposition in large-scale optimization [33] and nonlinear MPC [34]. A bipartite graph is one whose nodes can be

partitioned into two disjoint subsets such that each edge connects a node from one subset to a node in the other. In the context of optimization problems, one subset of nodes corresponds to the variables, while the other corresponds to the constraints. An edge is introduced between a variable node and a constraint node whenever the variable appears in that constraint [33]. The connectivity between the nodes of the graph is captured by the adjacency matrix $\mathbf{A}_d \in \{0, 1\}^{|\mathcal{V}| \times |\mathcal{V}|}$, where $(\mathbf{A}_d)_{ij} = 1$ if an edge exists between nodes i and j , and 0 otherwise.

The nodes (variable and constraint) and the edges of the bipartite graph can be augmented with features that encode additional information about the optimization problem. These node features are collected in a node feature matrix $\mathbf{X}_f \in \mathbb{R}^{|\mathcal{V}| \times d_0}$, where d_0 denotes the node-feature dimension. Edges may include non-zero coefficients of the constraint matrix as features, which are stored in an edge feature matrix $\mathbf{X}_e \in \mathbb{R}^{|\mathcal{E}| \times d_e}$, where d_e denotes the edge-feature dimension. Together, the matrices $\mathbf{A}_d$, $\mathbf{X}_f$, and $\mathbf{X}_e$ provide a compact way to represent the bipartite graph of an optimization problem.

2.4 Graph Neural Networks

Graph Neural Networks (GNNs) are a class of neural networks designed to operate on graph-structured data. Given a graph $\mathcal{G} = (\mathcal{V}, \mathcal{E})$, a GNN learns permutation-invariant functions that capture both local and global structure. Most GNN architectures follow a message-passing paradigm, where each node updates its representation by aggregating information from its neighbors. Common examples include the Graph Convolutional Network (GCN) [35] and the Edge-Conditioned Convolution (ECC) [36].

Of particular relevance to this work is the ECC, which extends GNNs to incorporate edge features into the message-passing process. Let $\mathbf{X}_f$ and $\mathbf{X}_e$ denote the node-feature and edge-feature matrices, respectively, and $\mathcal{N}(i)$ denote the closed neighborhood of i , i.e., $\mathcal{N}(i) := \{j \mid (j, i) \in \mathcal{E}\} \cup \{i\}$, as defined by the adjacency matrix $\mathbf{A}_d$. The ECC update rule is expressed as [36]:

$$\mathbf{X}_f^{(0)} = \mathbf{X}_f, \tag{6}$$

$$\mathbf{X}_f^{(\ell)}(i) = \frac{1}{|\mathcal{N}(i)|} \sum_{j \in \mathcal{N}(i)} \mathcal{F}^{(\ell)}(\mathbf{X}_e(j, i); \mathbf{w}^{(\ell)}) \mathbf{X}_f^{(\ell-1)}(j) + \mathbf{b}^{(\ell)}, \quad \ell = 1, \dots, L, \tag{7}$$

where $\mathbf{X}_f^{(\ell)} \in \mathbb{R}^{|\mathcal{V}| \times d_\ell}$ denotes the node-embedding matrix at layer ℓ , with d_ℓ denoting the node-embedding dimension, and $\mathbf{X}_f^{(\ell)}(i) \in \mathbb{R}^{d_\ell}$ denotes the embedding of node i (i.e., the i -th row of

$\mathbf{X}_f^{(\ell)}$). In addition, $\mathbf{X}_e(j, i) \in \mathbb{R}^{d_e}$ denotes the edge-feature vector associated with edge (j, i) . The filter-generating network $\mathcal{F}^{(\ell)}(\cdot; \mathbf{w}^{(\ell)}) : \mathbb{R}^{d_e} \rightarrow \mathbb{R}^{d_\ell \times d_{\ell-1}}$ maps edge features $\mathbf{X}_e(j, i)$ to a weight matrix, i.e., $\mathcal{F}^{(\ell)}(\mathbf{X}_e(j, i); \mathbf{w}^{(\ell)}) \in \mathbb{R}^{d_\ell \times d_{\ell-1}}$, with $d_{\ell-1}$ denoting the node-embedding dimension at layer $\ell - 1$. Furthermore, $\mathbf{b}^{(\ell)}$ is a bias term. The node embeddings at the final layer, $\mathbf{X}_f^{(L)}$, can serve as inputs to another MLP that produces task-specific outputs. Depending on the learning objective, GNNs can perform tasks (e.g., classification, regression) at the node level, edge level, or graph level. For graph-level tasks, pooling and readout operations are combined with the GNN layers to aggregate the node embeddings into a fixed-size graph representation. An important property of GNN architectures is their ability to process graphs of different sizes and topologies using the same set of learned parameters. This makes GNNs suitable for applications involving evolving graph structures, such as the bipartite graph representation of the master problem in GBD, where the addition of cuts at each iteration yields a graph whose structure evolves across iterations.

2.5 Reinforcement Learning (RL)

RL [37] is a machine learning paradigm in which an agent learns to make decisions by interacting with an environment and seeks to maximize cumulative rewards over time. The standard formalism for RL is the Markov Decision Process (MDP), which provides a mathematical framework for modeling sequential decision-making problems. A finite MDP is defined by a tuple $(\mathcal{S}, \mathcal{A}, \mathcal{P}, \mathcal{R})$, where $\mathcal{S}$ is a finite set of states, $\mathcal{A}$ is a finite set of actions, $\mathcal{P}(s' | s, a)$ defines the probability of moving to state s' and receiving a reward r when action a is taken in state s , and $\mathcal{R}$ is the reward function. At each time step t , the agent observes a state $s_t \in \mathcal{S}$, selects an action $a_t \in \mathcal{A}$, and receives a scalar reward r_t , while the environment transitions to a new state s_{t+1} according to $\mathcal{P}$. The agent’s objective is to learn a policy $\pi : \mathcal{S} \rightarrow \mathcal{A}$ that maximizes the expected cumulative discounted reward $\mathbb{E} \left[\sum_{t=0}^{\infty} \gamma^t r_t \right]$, where $\gamma \in [0, 1)$ is the discount factor.

RL methods are generally classified into value-based and policy-based methods. Value-based methods aim to learn a value function that estimates the expected return under a given policy. The optimal policy is then derived indirectly by selecting actions that maximize the learned value function. Policy-based methods, in contrast, directly optimize a parameterized policy via gradient ascent on the expected return. Central to this work are actor-critic methods, such as Proximal Policy Optimization (PPO) [38], which combine the strengths of value-based and policy-based paradigms. These methods maintain two distinct models: an actor, which represents a parameter-

ized policy, and a critic, which estimates a value function to guide the actor’s updates.

2.6 Imitation Learning (IL)

IL is a learning paradigm in which an agent seeks to learn a policy by mimicking expert behavior. Formally, the IL problem shares structural similarities with RL. However, the key distinction lies in the absence of an explicit reward signal. Rather, the agent is provided with a collection of expert trajectories ξ , where each trajectory $\xi = \{(s_0, a_0), (s_1, a_1), \dots\}$ comprises state-action pairs sampled from an expert policy π^* . The objective of IL is to recover a policy π that approximates the behavior of the expert.

Relevant to this work is behavioral cloning [39], which formulates the IL task as a supervised learning problem. Behavioral cloning is simple to implement and effective in settings with sufficient and representative expert data. In behavioral cloning, the agent is trained to minimize the discrepancy between its predicted actions and those of the expert. Given a data set of expert demonstrations Ξ , the objective is to solve:

$$\hat{\pi}^* \in \arg \min_{\pi} \sum_{\xi \in \Xi} \sum_{s \in \xi} \mathcal{L}(\pi(s), \pi^*(s)),$$

where $\mathcal{L}$ is a suitable loss function, $\pi^*(s)$ denotes the action of the expert in the state s , and $\hat{\pi}^*$ is the learned policy that imitates the expert.

3 Graph-Based Agent

We present the proposed graph-based agent to solve the master problem. The agent takes a bipartite graph representation of the master problem as its input and predicts the candidate values of the binary variables in the master problem. The development of the agent follows a two-stage approach, which integrates IL and RL. In the first stage, the agent is trained through IL to mimic an MIP solver. The IL agent policy then serves as the initial policy within an RL framework. In the second stage, this policy is fine-tuned through interactions with the GBD environment.

The remainder of this section is organized as follows: first, we present the bipartite graph of the master problem; second, we describe the architecture and training procedure employed in the IL stage; and finally, we outline the MDP formulation used for RL training.

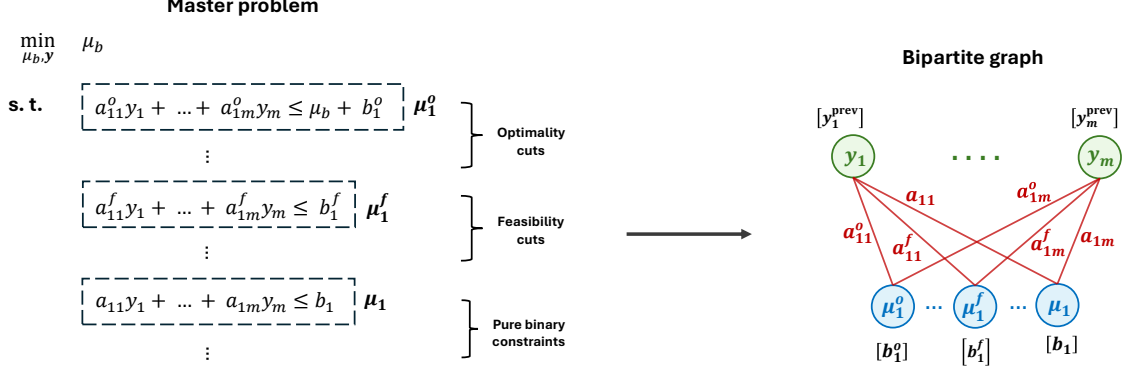


Figure 2: Bipartite graph of the master problem. The constraints are linear because the pure binary constraints are linear and the binary variables enter the objective and constraints of the original MINLP linearly.

3.1 Bipartite Graph Representation of the Master Problem

Each instance of the master problem is represented as a bipartite graph, following the general framework introduced in Section 2.3. Figure 2 illustrates a generic master problem instance and its graph encoding. The key components of the graph are summarized below.

- (i) **Variable nodes:** Each variable node corresponds to a binary variable in the master problem. The continuous variable μ_b , which appears in the optimality cuts, is omitted because including it in the graph was found to consistently reduce predictive accuracy in the IL stage.
- (ii) **Variable node features:** The value of each binary variable from the previous master problem iteration is used as the variable node feature. This choice is consistent with related studies, such as [32]. In Figure 2, these features are denoted as $[y_1^{\text{prev}}, \dots, y_m^{\text{prev}}]$.
- (iii) **Constraint nodes:** Each constraint node represents a constraint in the master problem, including feasibility cuts (μ_k^f for $k \in K_F$), optimality cuts (μ_k^o for $k \in K_O$), and pure-binary constraints (μ_t for $t \in \{1, \dots, o\}$), as illustrated in Figure 2.
- (iv) **Constraint node features:** The constant term on the RHS of each constraint (i.e., the optimality cuts, feasibility cuts, and pure-binary constraints) serves as the feature of the corresponding constraint node. In Figure 2, the constraint-node features of the optimality cuts, feasibility cuts, and pure-binary constraints are denoted as b_k^o for $k \in K_O$, b_k^f for $k \in K_F$, and b_t for $t \in [1, \dots, o]$, respectively. Given formulation (1), as well as the expressions for the optimality cuts (inequality (3)), feasibility cuts (inequality (4)), and pure-binary constraints (inequality (5)), the various constraint-node features can be obtained as follows.

For the *optimality cuts*,

$$\mu_b \geq f(\mathbf{x}^k) + \mathbf{e}^\top \mathbf{y} + \boldsymbol{\lambda}_k^\top (\mathbf{h}(\mathbf{x}^k) + \mathbf{A}\mathbf{y}) + \boldsymbol{\mu}_k^\top (\mathbf{g}(\mathbf{x}^k) + \mathbf{B}\mathbf{y}), \quad \forall k \in K_O,$$

the corresponding constraint-node feature is

$$b_k^o = - \left[f(\mathbf{x}^k) + \boldsymbol{\lambda}_k^\top \mathbf{h}(\mathbf{x}^k) + \boldsymbol{\mu}_k^\top \mathbf{g}(\mathbf{x}^k) \right], \quad \forall k \in K_O,$$

where $\mathbf{x}^k$ is an optimal solution of the subproblem $\mathcal{S}(\mathbf{y}^k)$ when it is feasible, and $\boldsymbol{\lambda}_k$ and $\boldsymbol{\mu}_k$ are the dual multipliers associated with the equality and inequality constraints of subproblem $\mathcal{S}(\mathbf{y}^k)$.

For the *feasibility cuts*,

$$\bar{\boldsymbol{\lambda}}_k^\top (\mathbf{h}(\bar{\mathbf{x}}^k) + \mathbf{A}\mathbf{y}) + \bar{\boldsymbol{\mu}}_k^\top (\mathbf{g}(\bar{\mathbf{x}}^k) + \mathbf{B}\mathbf{y}) \leq 0, \quad \forall k \in K_F,$$

the corresponding constraint-node feature is

$$b_k^f = - \left[\bar{\boldsymbol{\lambda}}_k^\top \mathbf{h}(\bar{\mathbf{x}}^k) + \bar{\boldsymbol{\mu}}_k^\top \mathbf{g}(\bar{\mathbf{x}}^k) \right], \quad \forall k \in K_F,$$

where $\bar{\mathbf{x}}^k$ is an the optimal solution of the feasibility subproblem $\mathcal{F}(\mathbf{y}^k)$, and $\bar{\boldsymbol{\lambda}}_k$ and $\bar{\boldsymbol{\mu}}_k$ are the respective dual multipliers associated with the equality and inequality constraints of the subproblem $\mathcal{F}(\mathbf{y}^k)$.

For the *pure-binary constraints*, the constraint-node features correspond directly to the entries of the vector $\mathbf{b}$ in inequality (5).

Collectively, the constraint-node features are $\{b_k^o\}_{k \in K_O}$, $\{b_k^f\}_{k \in K_F}$, and $\{b_t\}_{t=1}^o$, which together form the constraint-feature rows in $\mathbf{X}_f$.

- (v) **Edges:** An edge exists between a variable node and a constraint node if the corresponding binary variable appears in that constraint. Connectivity among nodes is captured by the adjacency matrix $\mathbf{A}_d$.
- (vi) **Edge features:** Each edge feature corresponds to the nonzero coefficient of a binary variable in the associated constraint. In Figure 2, the edge features of the optimality cuts, feasibility cuts, and pure-binary constraints are denoted as a_{kj}^o , a_{kj}^f , and a_{tj} , respectively. Given formu-

lation (1), as well as the expressions of the optimality cuts, feasibility cuts, and pure-binary constraints, the various edge features are derived as follows.

For the optimality cuts,

$$\mu_b \geq f(\mathbf{x}^k) + \mathbf{e}^\top \mathbf{y} + \boldsymbol{\lambda}_k^\top (\mathbf{h}(\mathbf{x}^k) + \mathbf{A}\mathbf{y}) + \boldsymbol{\mu}_k^\top (\mathbf{g}(\mathbf{x}^k) + \mathbf{B}\mathbf{y}), \quad \forall k \in K_O,$$

the edge feature linking variable y_j to the k th optimality cut is

$$a_{kj}^o = e_j + (\mathbf{A}^\top \boldsymbol{\lambda}_k)_j + (\mathbf{B}^\top \boldsymbol{\mu}_k)_j, \quad \forall j \in \{1, \dots, m\}, k \in K_O,$$

where e_j denotes the j -th entry of vector $\mathbf{e}$.

For the feasibility cuts,

$$\bar{\boldsymbol{\lambda}}_k^\top (\mathbf{h}(\bar{\mathbf{x}}^k) + \mathbf{A}\mathbf{y}) + \bar{\boldsymbol{\mu}}_k^\top (\mathbf{g}(\bar{\mathbf{x}}^k) + \mathbf{B}\mathbf{y}) \leq 0, \quad \forall k \in K_F,$$

the edge feature linking variable y_j to the k th feasibility cut is

$$a_{kj}^f = (\mathbf{A}^\top \bar{\boldsymbol{\lambda}}_k)_j + (\mathbf{B}^\top \bar{\boldsymbol{\mu}}_k)_j, \quad \forall j \in \{1, \dots, m\}, k \in K_F.$$

For the pure-binary constraints, the edge features correspond to the entries of $\mathbf{K}$ in inequality 5, that is,

$$a_{tj} = K_{tj}, \quad \forall j \in \{1, \dots, m\}, t \in \{1, \dots, o\}.$$

Collectively, the edge features are $\{a_{kj}^o\}_{k \in K_O}$, $\{a_{kj}^f\}_{k \in K_F}$, and $\{a_{tj}\}_{t=1}^o$ for all $j \in \{1, \dots, m\}$, which together form the edge-feature matrix $\mathbf{X}_e$.

Under this graph encoding, the number of variable nodes remains fixed across master problem instances, while the number of constraint nodes and edges grows over iterations as cuts are added. Consequently, the matrices $\mathbf{A}_d$, $\mathbf{X}_f$ and $\mathbf{X}_e$ grow in size as the GBD algorithm progresses. Furthermore, to enhance training stability and predictive performance, the entries of matrices $\mathbf{X}_f$ and $\mathbf{X}_e$ are normalized.

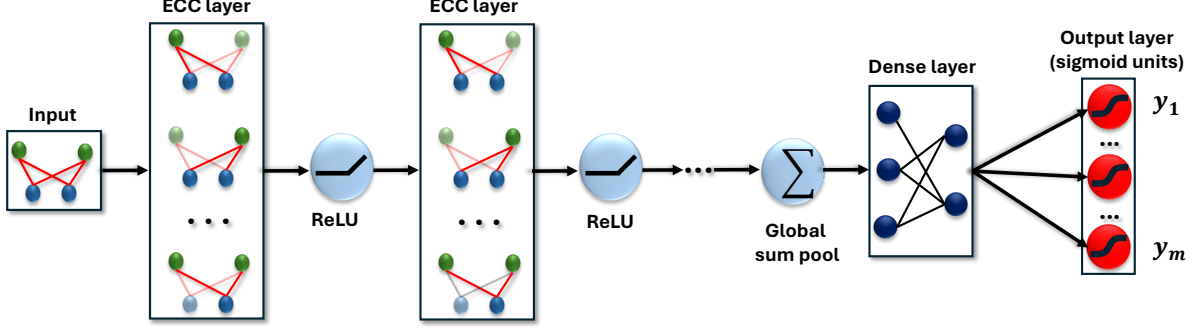


Figure 3: A schematic of the GNN with multi-headed sigmoid output layer.

3.2 IL Stage

The agent’s policy is parameterized by a GNN to handle the graph-structured nature of its inputs and to exploit the effectiveness of deep neural networks in approximating policies. The learning problem at this stage is formulated as a graph-level multi-label classification task. A schematic of the agent is shown in Figure 3, and its key components are described below:

- **Input layer:** The bipartite graph of the master problem.
- **Graph layer(s):** One or more ECC layers.
- **Readout layer:** A global sum pooling operation that aggregates node embeddings to produce a graph-level representation.
- **Dense layer(s):** Fully connected layers are applied after pooling to increase the network’s capacity to capture complex interactions.
- **Output layer:** A layer with as many units as binary variables in the master problem. Each unit uses a sigmoid activation function to produce a value in the range $[0, 1]$. This design yields a multi-headed output architecture, where each binary variable is predicted independently.

Following the specification of the agent’s architecture, it is trained using behavioral cloning. During training, the binary cross-entropy loss function is employed for the multi-label classification task. For a given set of ground truth labels $\mathbf{y}$ and predicted values $\hat{\mathbf{y}}$, the cross-entropy loss function $\mathcal{L}(\cdot)$ is defined as:

$$\mathcal{L}(\mathbf{y}, \hat{\mathbf{y}}) := \frac{1}{m} \sum_{i=1}^m [-y_i \log \hat{y}_i - (1 - y_i) \log(1 - \hat{y}_i)].$$

3.3 RL Stage

In this stage, the agent resulting from the first stage is fine-tuned within an actor–critic RL framework. The actor is initialized with the IL policy and therefore adopts the same architecture as the IL agent. To handle the graph-structured nature of its inputs and to exploit the effectiveness of deep neural networks for function approximation, the critic is also implemented as a GNN. Its architecture follows the same design as that of the actor (see Section 3.2) but differs in the output layer. Unlike the actor, which possesses a multi-headed output layer with sigmoid activation, the critic has a single-unit output layer without activation to approximate the state-value function. The key components of the MDP formulation that govern the agent’s decision-making process are described below.

3.3.1 State

The state of the agent is the bipartite graph of the master problem instance, which is compactly represented by the adjacency matrix $\mathbf{A}_d$, the node feature matrix $\mathbf{X}_f$, and the edge feature matrix $\mathbf{X}_e$. Together, $\mathbf{A}_d$, $\mathbf{X}_f$, and $\mathbf{X}_e$ define the state and serve as input to the actors and critic networks.

3.3.2 Action

The action $\mathbf{a}$ of the agent belongs to the discrete space $\{0, 1\}^m$, which corresponds to the m binary decision variables of the master problem. Each action therefore corresponds to a complete assignment of values to these binary decision variables. The design of the actor, particularly its output layer, yields a stochastic policy during training. Each unit of the actor’s output layer is equipped with a sigmoid activation, producing a value in the interval $[0, 1]$. This value is interpreted as the probability of assigning 1 to the corresponding binary variable. Thus, each output p_i defines a Bernoulli distribution, from which the binary decision can be sampled. The agent’s action $\mathbf{a}$ is obtained as a vector of independent samples from the Bernoulli distributions parameterized by $\mathbf{p}$. Given the policy π_θ , the action selection proceeds as follows:

$$\begin{aligned}\pi_\theta(\mathbf{A}_d, \mathbf{X}_f, \mathbf{X}_e) &:= \mathbf{p} := (p_1, \dots, p_m), \quad p_i \in [0, 1], \\ a_i &\sim \text{Bernoulli}(p_i), \quad i = 1, \dots, m, \\ \mathbf{a} &:= (a_1, \dots, a_m) \in \{0, 1\}^m.\end{aligned}\tag{8}$$

The Bernoulli output model used for action sampling in Equation (8) does not explicitly rep-

resent a joint distribution over all binary assignments. Nevertheless, this design choice does not imply that the binary variables are treated as independent within the policy network, since all probabilities p_i are computed from a shared graph-based representation of the master problem. This representation helps to capture the interactions among the binary variables induced by shared constraints. Alternatively, strongly coupled binary variables can be grouped and modeled jointly within each group. However, this can increase the complexity of the output layer compared to the multi-headed Bernoulli approach considered in this work.

3.3.3 Transition Dynamics

In iteration k , the current agent policy uses the matrices $\mathbf{A}_{d_k}$, $\mathbf{X}_{f_k}$, and $\mathbf{X}_{e_k}$ to produce action $\mathbf{a}_k$ according to Equation 8. However, $\mathbf{a}_k$ cannot be passed directly to the subproblem, as its feasibility with respect to the master problem is not guaranteed. Thus, $\mathbf{a}_k$ is first checked for feasibility. This check involves evaluating feasibility cuts (that is, the left-hand side (LHS) of inequality (4)) and the pure binary constraints (that is, the LHS of inequality (5)) using $\mathbf{a}_k$. If these constraints are satisfied, $\mathbf{a}_k$ is passed to the subproblem, which is solved to generate the appropriate cut. Conversely, if $\mathbf{a}_k$ is infeasible, the agent receives a penalty, and an MIP solver is invoked to solve the master problem. The resulting solution, $\bar{\mathbf{y}}_k$, is then passed to the subproblem. In either case, a transition occurs in which the master problem is updated by adding a cut, which modifies the matrices $\mathbf{A}_d$, $\mathbf{X}_f$, and $\mathbf{X}_e$ to yield $\mathbf{A}_{d_{k+1}}$, $\mathbf{X}_{f_{k+1}}$, and $\mathbf{X}_{e_{k+1}}$. In summary, under $\mathbf{a}_k$ (or $\bar{\mathbf{y}}_k$), the state of the master problem transitions from $(\mathbf{A}_{d_k}, \mathbf{X}_{f_k}, \mathbf{X}_{e_k})$ to $(\mathbf{A}_{d_{k+1}}, \mathbf{X}_{f_{k+1}}, \mathbf{X}_{e_{k+1}})$, which defines the dynamics of the agent’s environment.

3.3.4 Reward Design

The agent’s objective is to choose actions that (i) render the master problem feasible, (ii) improve the GBD bound gap, and (iii) reduce the solution time at each iteration. These objectives are captured by a hybrid reward function which is defined as:

$$r := \alpha_1 r_{\text{feas}} + \alpha_2 r_{\text{gap}} - \alpha_3 r_{\text{time}}, \quad (9)$$

where $\alpha_1, \alpha_2, \alpha_3 \geq 0$ are weighting coefficients that balance the contributions of the feasibility reward r_{feas} , the gap improvement reward r_{gap} , and the time penalty $-r_{\text{time}}$, respectively. The components of r are described below:

1. **Feasibility reward:** Penalizes infeasible actions and rewards feasible ones:

$$r_{\text{feas}} := \begin{cases} -\beta_1 & \text{if the master problem is infeasible} \\ \beta_2 & \text{if the master problem is feasible} \end{cases}, \quad (10)$$

where $\beta_1, \beta_2 \geq 0$.

2. **Gap improvement reward:** Encourages actions that reduce the GBD bound gap between iterations:

$$r_{\text{gap}} := \begin{cases} \max\left(0, \frac{\Delta_{k-1} - \Delta_k}{\Delta_0}\right) & \text{if the master problem is feasible,} \\ 0 & \text{otherwise} \end{cases}, \quad (11)$$

where $\Delta_k := \text{UBD}_k - \text{LBD}_k$ denotes the bound gap at iteration k , and Δ_0 is the initial gap. This reward is set to 0 when the agent’s assignment is infeasible, when it yields no reduction in the bound gap between two consecutive iterations, or when it worsens the bound gap (i.e., the gap increases). Otherwise, the reward equals the normalized reduction in the bound gap.

3. **Time penalty:** Discourages actions that prolong the solution time of the subproblem. Given that the solution of the master problem can affect the complexity of the subproblem, this term allows the framework to incorporate the efficiency of the subproblem as one of its objectives. Let t_{SP} denote the time required to solve the subproblem. To prevent excessively large values from dominating r , the penalty is clipped at a threshold τ :

$$r_{\text{time}} := \min(t_{\text{SP}}, \tau). \quad (12)$$

3.3.5 Training Phase

The agent is trained over a fixed number of episodes N_{ep} . Among the available actor-critic methods, we adopt the PPO algorithm [38] due to its robustness and stability in complex training environments. In PPO, the critic estimates the state-value function, while the actor is updated based on an *advantage* estimate, which measures how much better a selected action performs relative to the expected performance at the same state. Agent training proceeds by collecting and storing experience tuples of the form $(S_k, \mathbf{a}_k, r, S_{k+1})$, where $S_k := (\mathbf{A}_{\mathbf{d}_k}, \mathbf{X}_{\mathbf{f}_k}, \mathbf{X}_{\mathbf{e}_k})$ and $S_{k+1} := (\mathbf{A}_{\mathbf{d}_{k+1}}, \mathbf{X}_{\mathbf{f}_{k+1}}, \mathbf{X}_{\mathbf{e}_{k+1}})$. These stored tuples are then used to update the actor π_θ and

critic V_ϕ networks according to the PPO algorithm. The complete training loop for the agent within the GBD framework is summarized in Algorithm 2 of the Supplementary Material.

4 Verification and Coordination Mechanism

While the agent seeks to enhance the computational efficiency of GBD, relying solely on it to solve the master problem presents several challenges. First, although the agent’s hybrid reward encourages it to produce actions that are feasible with respect to the master problem, its actions are not guaranteed to be feasible. Therefore, a sole reliance on the agent could result in the passing of infeasible assignments to the subproblem. Second, the actions provided by the agent are not necessarily optimal. Consequently, suboptimal actions can yield a master-problem value μ_b , which we treat as a candidate LBD, that can exceed the incumbent UBD. In this case, the resulting value cannot be regarded as a valid lower bound for the original MINLP, though it should be noted that $\mu_b \leq \text{UBD}$ does not certify μ_b as a valid lower bound for the original MINLP.

To address these challenges, we propose a verification and coordination strategy that leverages the agent to quickly generate assignments for the binary decision variables, with fallbacks to an MIP solver whenever the proposed assignment fails feasibility checks or yields a master-problem value μ_b that exceeds the incumbent UBD. We refer to this approach as *confidence-based assignment*. The main steps are summarized in Algorithm 1, and the details are provided in the next section.

4.1 Confidence-Based Assignments

As discussed in Section 3.3.2, the agent’s policy produces a probability vector $\mathbf{p} = (p_1, \dots, p_m)$, where each p_i parameterizes a Bernoulli distribution for the corresponding binary decision variable y_i . During training, actions are sampled stochastically from these distributions to encourage exploration. However, during deployment, a deterministic strategy is adopted. To enhance reliability of the agent’s actions, the probabilities are post-processed using a confidence-based rule. Specifically, we define a high-confidence region as $[0, \delta_1] \cup [\delta_2, 1]$ with $0 \leq \delta_1 \leq \delta_2 \leq 1$. The probabilities within this region are fixed deterministically to 0 (if $p_i \leq \delta_1$) or 1 (if $p_i \geq \delta_2$). The probabilities in the remaining interval (δ_1, δ_2) are treated as uncertain, and the corresponding variables are left unfixed for the MIP solver to determine. This procedure yields three possible scenarios:

1. **Full Assignment:** All probabilities lie within the confidence region, so all binary variables are fixed deterministically.

Algorithm 1: Confidence-based assignment.

Input: $\mathbf{p} \in [0, 1]^m$, δ_1, δ_2 with $0 \leq \delta_1 \leq \delta_2 \leq 1$, UBD, $\{\mathcal{O}_k(\cdot)\}_{k \in K_O}$, ε

Output: $\mathbf{y} \in \{0, 1\}^m$, μ_b

Initialize $\mathbf{y}$ as free; $\mathcal{Y} \leftarrow \emptyset$

foreach $i \in \{1, \dots, m\}$ **do**

if $p_i \leq \delta_1$ **then**
 Set $y_i \leftarrow 0$; add i to $\mathcal{Y}$
 else
 if $p_i \geq \delta_2$ **then**
 Set $y_i \leftarrow 1$; add i to $\mathcal{Y}$
 else
 Leave y_i free

if $|\mathcal{Y}| = m$ **then**

 ▷ Full assignment

if $\mathbf{y}$ infeasible for master problem $\mathcal{M}$ **then**

 Solve master problem $\mathcal{M}$ to obtain $\bar{\mathbf{y}}, \bar{\mu}_b$
 Set $\mathbf{y} \leftarrow \bar{\mathbf{y}}$, $\mu_b \leftarrow \bar{\mu}_b$

else

if $K_O \neq \emptyset$ **then**
 Compute $\hat{\mu}_b \leftarrow \max_{k \in K_O} \mathcal{O}_k(\mathbf{y})$

else
 Set $\hat{\mu}_b \leftarrow -\infty$

if $\hat{\mu}_b \leq \text{UBD} + \varepsilon$ **then**

 Set $\mu_b \leftarrow \hat{\mu}_b$

else

 Solve master problem $\mathcal{M}$ (with $y_1, \dots, y_m$ free) to obtain $\bar{\mathbf{y}}, \bar{\mu}_b$
 Set $\mathbf{y} \leftarrow \bar{\mathbf{y}}$, $\mu_b \leftarrow \bar{\mu}_b$

else

if $0 < |\mathcal{Y}| < m$ **then**

 ▷ Partial assignment

 Solve master problem $\mathcal{M}$ with y_i fixed, $\forall i \in \mathcal{Y}$ to obtain $\bar{\mathbf{y}}, \bar{\mu}_b$

if master problem $\mathcal{M}$ infeasible **then**

 Solve master problem $\mathcal{M}$ (with $y_1, \dots, y_m$ free) to obtain $\bar{\mathbf{y}}, \bar{\mu}_b$
 Set $\mathbf{y} \leftarrow \bar{\mathbf{y}}$, $\mu_b \leftarrow \bar{\mu}_b$

else

if $\bar{\mu}_b \leq \text{UBD} + \varepsilon$ **then**

 Set $\mathbf{y} \leftarrow \bar{\mathbf{y}}$, $\mu_b \leftarrow \bar{\mu}_b$

else

 Solve master problem $\mathcal{M}$ (with $y_1, \dots, y_m$ free) to obtain $\bar{\mathbf{y}}, \bar{\mu}_b$
 Set $\mathbf{y} \leftarrow \bar{\mathbf{y}}$, $\mu_b \leftarrow \bar{\mu}_b$

else

 ▷ No assignment

 Solve master problem $\mathcal{M}$ (with $y_1, \dots, y_m$ free) to obtain $\bar{\mathbf{y}}, \bar{\mu}_b$

 Set $\mathbf{y} \leftarrow \bar{\mathbf{y}}$, $\mu_b \leftarrow \bar{\mu}_b$

return $(\mathbf{y}, \mu_b)$

2. **Partial Assignment:** A subset of variables is fixed deterministically, and the remaining ones are determined by the solver.
3. **No Assignment:** None of the probabilities meet the confidence threshold, so all variables are determined by the solver.

These are described in detail below.

4.1.1 Full Assignment

When all probabilities are within the high-confidence region, all binary variables are fixed to obtain a candidate assignment, denoted by $\hat{\mathbf{y}}$. Two checks are performed to assess its suitability.

1. **Feasibility:** The candidate $\hat{\mathbf{y}}$ is checked against the constraints of the master problem. This check is restricted to feasibility cuts and pure-binary constraints. Specifically, for each feasibility cut $\mathcal{C}_f^{(k)}$, $\forall k \in K_F$, and pure-binary constraint $\mathcal{C}_p$, the LHS is evaluated using $\hat{\mathbf{y}}$. If any of these constraints are violated, $\hat{\mathbf{y}}$ is deemed infeasible. In this case, the MIP solver is invoked to solve the master problem and the resulting solution is passed to the subproblem.
2. **Evaluation of master-problem value $\hat{\mu}_b$:** If $\hat{\mathbf{y}}$ passes the feasibility check, the next step is to compute the value of $\hat{\mu}_b$ associated with it. In this case, solving the master problem with $\hat{\mathbf{y}}$ can be avoided. Instead, $\hat{\mu}_b$ is obtained by evaluating the RHS of each optimality cut, $\mathcal{O}_k(\hat{\mathbf{y}})$, $\forall k \in K_O$, and taking the maximum:

$$\hat{\mu}_b := \max_{k \in K_O} \mathcal{O}_k(\hat{\mathbf{y}}).$$

Since the agent's assignments are not guaranteed to be optimal, the resulting value $\hat{\mu}_b$ is treated as a *candidate* LBD. To screen the assignment $\hat{\mathbf{y}}$, $\hat{\mu}_b$ is compared with the incumbent UBD, up to the tolerance ε . If $\hat{\mu}_b > \text{UBD} + \varepsilon$, the assignment is rejected and an MIP solver is invoked to solve the master problem (with all binary variables free); otherwise, $\hat{\mathbf{y}}$ is passed to the subproblem. This screening step filters out assignments that would necessarily yield invalid LBDs for the original MINLP; however, as clarified in Remark 1, it does not certify the optimality of $\hat{\mathbf{y}}$ or the validity of $\hat{\mu}_b$ as an LBD for the original MINLP.

4.1.2 Partial Assignment

When only a subset of probabilities lies within the high-confidence region, the corresponding variables are fixed, and an MIP solver is invoked to determine the remaining ones. If the fixed values render the master problem infeasible, the assignment is discarded, and the solver is invoked to resolve the master problem with all binary variables free. The resulting solution is then passed to the subproblem.

If the master problem is feasible for the fixed values, the solver returns a complete binary vector, denoted as $\bar{\mathbf{y}}$, together with the corresponding master-problem value $\bar{\mu}_b$, which is treated as a *candidate* LBD. To screen this candidate solution, $\bar{\mu}_b$ is compared with the incumbent UBD, up to the tolerance ε . If $\bar{\mu}_b \leq \text{UBD} + \varepsilon$, $\bar{\mathbf{y}}$ is accepted and forwarded to the subproblem; otherwise, the master problem is re-solved with all binary variables free, and the resulting solution is passed to the subproblem. While this screening step eliminates assignments that would necessarily yield invalid LBDs for the original MINLP, as clarified in Remark 1, it does not certify the optimality of $\bar{\mathbf{y}}$ or the validity of $\bar{\mu}_b$ as an LBD for the original MINLP.

Remark 1. *With numerical tolerance $\varepsilon \geq 0$, the test $\mu_b \leq \text{UBD} + \varepsilon$ used in the full (where $\mu_b = \hat{\mu}_b$) and partial (where $\mu_b = \bar{\mu}_b$) assignment cases is a screening step, and not a certification of (i) the optimality of the solution that yields μ_b or (ii) the validity of μ_b as a lower bound for the original MINLP. In particular, if z^* denotes the optimal objective value of the original MINLP, the test ensures that $\mu_b > \text{UBD} + \varepsilon \implies \text{reject the agent's assignment}$. However, the converse does not hold. That is, $\mu_b \leq \text{UBD} + \varepsilon \not\implies \mu_b \leq z^*$. Thus, the screening test can accept assignments for which μ_b is not a valid lower bound for the original MINLP. A stronger certification can be obtained by solving the master problem under a limited computational budget using the accepted assignment as a warm start, and using the solver-reported best bound as a certified lower bound.*

4.1.3 No Assignment

When none of the probabilities lies within the high-confidence region, an MIP solver is invoked to solve the master problem directly. The resulting solution is then passed to the subproblem.

4.2 Graph-based IL and RL for the GBD Algorithm

Algorithm 2 presents the proposed graph-based IL and RL for the GBD algorithm, which results from incorporating the trained agent into the classical GBD loop via the verification and coordina-

tion mechanism in Algorithm 1. In the proposed approach, the master problem is not necessarily solved to optimality at each iteration, particularly in the full and partial assignment cases where the agent’s actions are accepted by Algorithm 1. Consequently, the values of μ_b^k obtained from Algorithm 1 over iterations, which serve as candidate lower-bound values, are not expected to form a monotone nondecreasing sequence as in classical GBD. This can lead to oscillations in the candidate sequence and make it difficult to track the progress of Algorithm 2.

To enable progress tracking, we maintain a nondecreasing record of the candidate lower-bound values. Let μ_b^k denote the candidate value returned at iteration k by Algorithm 1, and define the monotone record CLBD_k by

$$\text{CLBD}_k := \max \left\{ \mu_b^k, \text{CLBD}_{k-1} \right\}, \quad \text{CLBD}_0 = -\infty.$$

We also define a surrogate bound gap using the incumbent UBD and the monotone record of candidate LBDs, and use it as the stopping criterion for Algorithm 2. Specifically, with UBD_k denoting the incumbent upper bound at iteration k , we terminate when

$$\text{UBD}_k - \text{CLBD}_k \leq \varepsilon,$$

for a user-defined tolerance $\varepsilon > 0$.

4.2.1 Convergence Properties

Algorithm 2 shares similarities with inexact GBD methods, in which the master problem is solved approximately. In such inexact GBD approaches, convergence guarantees can be established, under standard assumptions, by solving the master problem to optimality at selected iterations (typically fewer iterations) [8, 40]. In the proposed approach, the parameters δ_1 and δ_2 that define the confidence region in Algorithm 1 can be viewed as tunable thresholds that govern when the master problem is solved approximately (via the assignments produced by the agent) and when it is solved to optimality (via the no-assignment case). By scheduling these thresholds so that the no-assignment case is invoked at selected iterations, Algorithm 2 admits, in principle, the convergence guarantees established for inexact GBD approaches. We do not provide a detailed study of threshold scheduling strategies in this work, and leave this topic for future work.

Algorithm 2: Graph-based agent integration into GBD.

Input: $\mathbf{y}^0, \varepsilon, \pi_\theta, \delta_1, \delta_2$
Initialize: $k \leftarrow 0, \text{CLBD} \leftarrow -\infty, \text{UBD} \leftarrow +\infty, K_O \leftarrow \emptyset, K_F \leftarrow \emptyset$
while $\text{UBD} - \text{CLBD} > \varepsilon$ **do**
 Solve subproblem $\mathcal{S}(\mathbf{y}^k)$
 if *subproblem $\mathcal{S}(\mathbf{y}^k)$ is feasible* **then**
 Update $\text{UBD} \leftarrow \min(\text{UBD}, Z(\mathbf{y}^k))$
 Add $\mathcal{C}_o^{(k)}$ to master problem $\mathcal{M}$
 Update $K_O \leftarrow K_O \cup \{k\}$
 else
 Solve subproblem $\mathcal{F}(\mathbf{y}^k)$
 Add $\mathcal{C}_f^{(k)}$ to master problem $\mathcal{M}$
 Update $K_F \leftarrow K_F \cup \{k\}$
 end
Construct $\mathcal{G}_k = (\mathbf{A}_{d_k}, \mathbf{X}_{f_k}, \mathbf{X}_{e_k})$ of master problem $\mathcal{M}$
Obtain probabilities $\mathbf{p} \leftarrow \pi_\theta(\mathbf{A}_{d_k}, \mathbf{X}_{f_k}, \mathbf{X}_{e_k})$
Invoke Algorithm 1 with $\mathbf{p}, \delta_1, \delta_2, \text{UBD}, \{\mathcal{O}_k(\mathbf{y})\}_{k \in K_O}, \varepsilon$ to obtain $\mathbf{y}^{k+1}, \mu_b^k$
Update $\text{CLBD} \leftarrow \max(\text{CLBD}, \mu_b^k), \mathbf{y}^k \leftarrow \mathbf{y}^{k+1}, k \leftarrow k + 1$
end

5 Case Studies

Two case studies are presented to demonstrate the performance of the proposed approach. The first is a generic MINLP that provides a controlled setting to evaluate the specific design choices of the proposed method. The second focuses on a real application, where the proposed approach is applied to an irrigation scheduling problem.

5.1 Case Study 1

First, we consider the following problem adapted from [31] denoted as $\mathcal{E}$:

$$\begin{aligned}
 \min_{\mathbf{x}, \mathbf{y}} \quad & 5y_1 + 8y_2 + 6y_3 + 10y_4 + 6y_5 - 10x_3 - 15x_5 - 15x_9 + 15x_{11} + 5x_{13} - 20x_{16} \\
 & + \exp(x_3) + \exp\left(\frac{x_5}{1.2}\right) - 60 \ln(x_{11} + x_{13} + 1) + 140
 \end{aligned} \tag{E1}$$

$$\text{s.t.} \quad -\ln(x_{11} + x_{13} + 1) \leq 0 \tag{E2}$$

$$-x_3 - x_5 - 2x_9 + x_{11} + 2x_{16} \leq 0 \tag{E3}$$

$$-x_3 - x_5 - 0.75x_9 + x_{11} + 2x_{16} \leq 0 \tag{E4}$$

$$x_9 - x_{16} \leq 0 \tag{E5}$$

$$2x_9 - x_{11} - 2x_{16} \leq 0 \tag{E6}$$

$$-0.5x_{11} + x_{13} \leq 0 \quad (\text{E7})$$

$$0.2x_{11} - x_{13} \leq 0 \quad (\text{E8})$$

$$\exp(x_3) - Uy_1 \leq 1 \quad (\text{E9})$$

$$\exp\left(\frac{x_5}{1.2}\right) - Uy_2 \leq 1 \quad (\text{E10})$$

$$1.25x_9 - Uy_3 \leq 0 \quad (\text{E11})$$

$$x_{11} + x_{13} - Uy_4 \leq 0 \quad (\text{E12})$$

$$-2x_9 + 2x_{16} - Uy_5 \leq 0 \quad (\text{E13})$$

$$y_1 + y_2 = 1 \quad (\text{E14})$$

$$y_4 + y_5 \leq 1 \quad (\text{E15})$$

$$\mathbf{y}^T = (y_1, y_2, y_3, y_4, y_5) \in \{0, 1\}^5, \quad \mathbf{x}^T = (x_3, x_5, x_9, x_{11}, x_{13}, x_{16}) \in \mathbb{R}^6$$

$$\mathbf{a}^T = (0, 0, 0, 0, 0, 0), \quad \mathbf{b}^T = (2, 2, 2, -, -, 3), \quad \mathbf{a} \leq \mathbf{x} \leq \mathbf{b}, \quad U = 10$$

5.1.1 Simulation Settings

5.1.1.1 Data Generation

Several components of problem $\mathcal{E}$ were parameterized to produce varied solutions across simulation runs. In particular, the cost function was expressed as

$$c_1y_1 + c_2y_2 + c_3y_3 + c_4y_4 + c_5y_5 + F_{\mathcal{E}}(\mathbf{x}),$$

where $F_{\mathcal{E}}(\mathbf{x})$ compactly denotes all $\mathbf{x}$ -dependent terms in Equation (E1). The coefficients c_1, c_2, c_3, c_4 were sampled as positive integers from the interval $[1, 39]$, while c_5 was also sampled as a positive integer from $[1, 7]$. These ranges were selected based on a series of simulation experiments, which indicated that they yield parameter realizations with varied optimal solutions.

Before training the agent, 3,000 distinct realizations of the coefficient vector $(c_1, \dots, c_5)$ were generated. For each parameter set, GBD was used to solve the corresponding instance. The specific master problem and subproblem formulations used during this process are provided in Section 1.1 of the Supplementary Information. The master problem was solved using Gurobi 12.0.1 [41], while the subproblem was solved using IPOPT 3.12.6 [42].

5.1.1.2 Agent Training

The dataset from Section 5.1.1.1 was used to train the agent in the IL stage. The model architecture and hyperparameters are summarized in Table 2 of the Supplementary Material. Graph operations were implemented using the Spektral library [43], and training was carried out in Keras [44]. In the RL stage, training was performed over 10,000 episodes, each corresponding to a distinct parameter realization. The critic network architecture, reward design parameters, and RL training hyperparameters are presented in Table 3 of the Supplementary Material. The training of the model in the IL and RL stages took 20 minutes and 3 hours, respectively on an Intel® Core™ i7-14700 processor.

5.1.1.3 Performance Evaluation

In addition to the proposed approach, four baselines were considered: (i) classical GBD using the feasibility pump heuristic [45] during the solution of the master problem; (ii) classical GBD, where the agent’s outputs are rounded to their nearest binary elements and supplied to the solver as variable hints; (iii) an IL agent; and (iv) an RL agent with a randomly initialized actor network. For evaluation, a comparative study was conducted using 100 new parameter realizations. All agent-based methods (IL agent, randomly initialized RL agent, and the proposed approach) were implemented in accordance with Algorithm 2. The confidence thresholds were set at $\delta_1 = 0.10$ and $\delta_2 = 0.90$. These values were manually tuned to strike a balance between computational efficiency and the selection of assignments with low uncertainty. For reference, classical GBD was also applied to the same 100 problem instances. Performance was assessed based on runtime and agreement with the reference objective value obtained by directly solving the original MINLP. Additionally, the agent-based methods were evaluated based on the frequency with which their assignments were accepted by Algorithm 1.

5.1.2 Results and Discussion

Table 1 summarizes the runtime performance of the six approaches. Both the proposed approach and the IL agent reduced the master problem runtime relative to classical GBD, with reductions of 56% and 42%, respectively. By contrast, the RL agent with a randomly initialized actor increased master problem runtime by 4%, indicating that its assignments often prolong the solution process. In terms of total GBD runtime, the proposed method achieved a 42% reduction and the IL agent

Table 1: Runtime performance across methods, over 100 test instances. Values are reported as mean $\pm$ standard deviation over the 100 instances. Percentages in parentheses indicate improvement relative to classical GBD.

Method	Master problem runtime (s)	Total GBD runtime (s)
Classical GBD	0.54 ± 0.13	0.64 ± 0.13
Classical GBD + feasibility pump heuristic	0.48 ± 0.11 (11%)	0.56 ± 0.13 (13%)
Classical GBD + agent outputs as variable hints	0.57 ± 0.14 (-6%)	0.67 ± 0.13 (-5%)
IL Agent	0.31 ± 0.12 (42%)	0.44 ± 0.12 (31%)
Proposed	0.24 ± 0.11 (56%)	0.37 ± 0.12 (42%)
RL (Random Init)	0.56 ± 0.31 (-4%)	0.66 ± 0.33 (-3%)

achieved a 31% reduction, whereas the randomly initialized RL agent increased total time by 3%. These results are consistent with Figure 4, which shows that, aggregated over all binary variables across the 100 instances, the proposed method fixed 84% of binary variables, compared to 78.8% for the IL agent and 32.7% for the randomly initialized RL agent.

These findings support the hypothesis that combining IL and RL yields greater computational benefits in the combinatorial setting of the master problem than using IL alone or RL alone. In the proposed approach, the RL stage refines the IL policy, leading to more confident and higher-quality binary assignments, as reflected in Figure 4. By contrast, the randomly initialized RL agent struggles in the complex GBD environment, producing fewer suitable assignments and triggering more frequent solver calls. This limitation is also reflected in Table 2, where the randomly initialized RL agent produces objective values that do not match the reference objective value in 30 of 100 instances. Across the full test set, this corresponds to a relative cost gap of 0.09 ± 0.16 . In contrast, both the IL agent and the proposed method match the reference objective value in all 100 instances (within solver tolerances).

Furthermore, incorporating the feasibility pump heuristic reduced the master problem runtime by 11%, which translated into a 13% reduction in total GBD runtime. This confirms that the proposed approach provides larger speedups than the feasibility pump baseline, both for the master problem and the overall GBD loop. In contrast, using rounded agent outputs as variable hints performed worse than both the proposed approach and classical GBD. In particular, this method increased the master problem and total runtimes by 6% and 5%, respectively. Since the rounded agent outputs are not guaranteed to be feasible or near-optimal with respect to the current master problem, they may provide misleading guidance and increase the solver’s search effort. However, across all test instances, the feasibility pump and variable hints variants yield objective values that match the reference objective value (within solver tolerances).

Table 2: Comparison to the reference solution obtained by directly solving the original MINLP, over 100 test instances. The second column reports the number of instances for which the final objective value matches the reference objective value, within solver tolerances. Relative cost gap values are reported as mean $\pm$ standard deviation.

Method	Objective match/Total	Relative cost gap
Classical GBD	100/100	0.00 $\pm$ 0.00
Classical GBD + feasibility pump heuristic	100/100	0.00 $\pm$ 0.00
Classical GBD + agent outputs as variable hints	100/100	0.00 $\pm$ 0.00
IL Agent	100/100	0.00 $\pm$ 0.00
Proposed	100/100	0.00 $\pm$ 0.00
RL (Random Init)	70/100	0.09 $\pm$ 0.16

5.2 Case Study 2: Closed-loop Irrigation Scheduling

In this case study, the proposed approach is applied to a closed-loop irrigation scheduling problem formulated as a mixed-integer MPC (MIMPC). The MIMPC scheduler was developed in [46], where it demonstrated improvements in water use efficiency compared to conventional scheduling methods. Despite these benefits, computational efficiency remained a key bottleneck. Since the scheduler must be solved on a daily basis to account for weather forecast uncertainty and because scaling to larger fields further increases complexity, decomposition methods such as GBD were recommended as a more efficient solution method. We apply the proposed approach to the MIMPC irrigation scheduler and evaluate its potential to improve computational efficiency without compromising irrigation performance.

In irrigation scheduling, the goal is to determine when to irrigate within a scheduling horizon and how much water to apply on those days to ensure optimal crop development. Formally, the scheduling problem is stated as follows:

Given:

- **Scheduling Horizon (N):** Number of days over which irrigation decisions are to be made.
- **Weather Predictions:** Daily forecasts of reference evapotranspiration (ET_0) and precipitation (R_n), where ET_0 quantifies combined water loss due to evaporation and plant transpiration, and R_n represents natural water input into the soil.
- **Crop Information:** Crop coefficient (K_c) values used to scale ET_0 to reflect crop-specific water requirements at different stages of growth.
- **Soil Moisture Content:** Initial spatial distribution of soil moisture (θ) in the field.

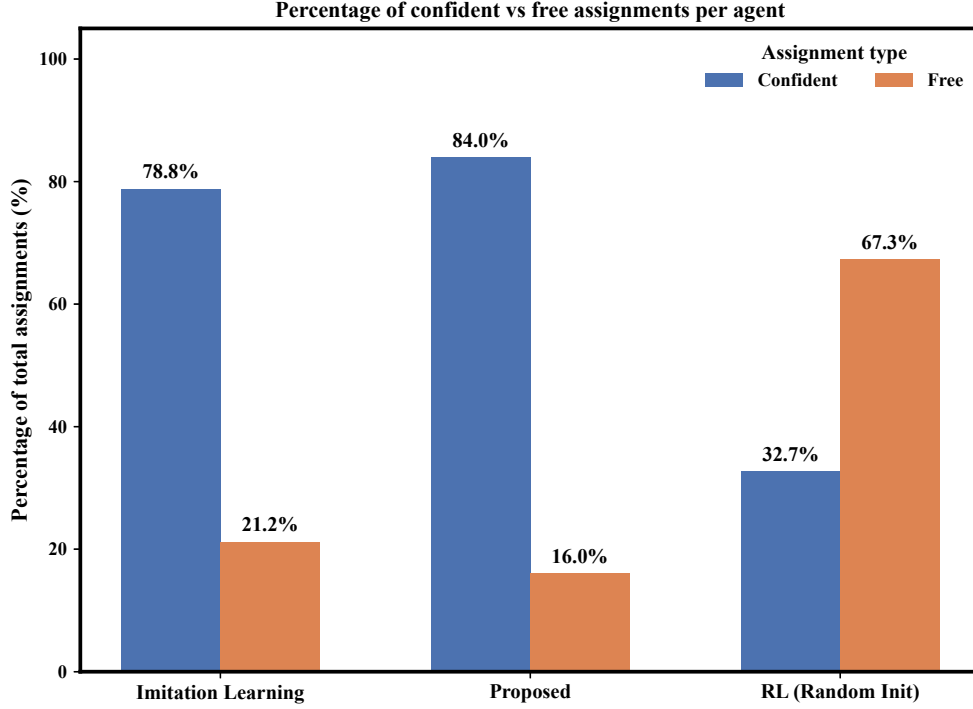


Figure 4: Percentage of confident and free binary variable assignments produced by each agent-based method over 100 test instances.

Determine:

- **Irrigation Timing:** The days within the scheduling horizon on which the irrigation is to be performed.
- **Irrigation Amounts:** Daily water application quantities (u) during each irrigation event, which is constrained to be zero on non-irrigation days and strictly positive on irrigation days.

In the MIMPC formulation, the irrigation timing is modeled with binary decision variables. Specifically, for each day on the horizon, a binary variable c_j is introduced which is equal to one if irrigation is performed on day j and zero otherwise. The MIMPC scheduler employs a predictive soil moisture model to generate irrigation schedules that maintain soil moisture θ within a zone that is known to promote optimal crop development. In addition, the formulation minimizes the operating costs of irrigation events and the amount of water applied. For day d , the scheduler $\mathcal{I}(\theta)$ is given by:

$$\min_{\bar{\epsilon}, \underline{\epsilon}, \mathbf{u}, \mathbf{c}} \underbrace{\sum_{j=d+1}^{d+N} [\bar{Q}\bar{\epsilon}_j^2 + \underline{Q}\underline{\epsilon}_j^2]}_{\text{Zone tracking}} + \underbrace{\sum_{j=d}^{d+N-1} R_c c_j}_{\text{Irrigation cost}} + \underbrace{\sum_{j=d}^{d+N-1} R_u u_j}_{\text{Irrigation volume cost}} \quad (13a)$$

$$\text{s.t. } \theta_{j+1} = \mathcal{F}_m(\theta_j, K_{c_j}, \text{ET}_{0_j}, R_{n_j}, u_j), \quad j \in [d, d+N-1] \quad (13b)$$

$$\theta_d = \theta(d), \quad (13c)$$

$$\underline{\nu} - \underline{\epsilon}_j \leq \theta_j \leq \bar{\nu} + \bar{\epsilon}_j, \quad j \in [d+1, d+N] \quad (13d)$$

$$c_j \underline{u} \leq u_j \leq c_j \bar{u}, \quad j \in [d, d+N-1] \quad (13e)$$

$$c_j \in \{0, 1\}, \quad j \in [d, d+N-1] \quad (13f)$$

$$\underline{\epsilon}_j \geq 0, \quad \bar{\epsilon}_j \geq 0, \quad j \in [d+1, d+N] \quad (13g)$$

where $\bar{\epsilon} := [\bar{\epsilon}_{d+1}, \dots, \bar{\epsilon}_{d+N}]$, $\underline{\epsilon} := [\underline{\epsilon}_{d+1}, \dots, \underline{\epsilon}_{d+N}]$, $\mathbf{c} := [c_d, \dots, c_{d+N-1}]$, and $\mathbf{u} := [u_d, \dots, u_{d+N-1}]$.

The variables $\underline{\epsilon}_j$ and $\bar{\epsilon}_j$ are nonnegative slack variables introduced to relax, respectively, the lower $\underline{\nu}$ and upper $\bar{\nu}$ bounds of the target zone, with $\underline{Q}$ and $\bar{Q}$ denoting their per-unit penalty costs. The parameters R_u and R_c correspond to the per-unit cost of the applied irrigation amount and the fixed operational cost of the irrigation system, respectively.

The constraint (13c) specifies the initial state, which is assumed to be known and obtained from the final soil moisture estimate produced in an offline estimation step. The constraint (13b) represents the dynamics of soil moisture, initially described by a well-calibrated mechanistic model based on the Richards' equation. The calibration uses field-specific parameters determined through an offline parameter estimation procedure. Due to the high computational burden of the mechanistic model, a long short-term memory (LSTM) network, which was trained according to the approach proposed in [46], is adopted as a surrogate to approximate the calibrated dynamics. The constraint (13e) enforces operational limits on irrigation amounts such that if $c_j = 0$, the applied volume is fixed at zero. Conversely, if $c_j = 1$, the applied amount is constrained between $\underline{u}$ and $\bar{u}$.

The MIMPC scheduler was applied to a section of a large-scale agricultural field at a Research Farm operated by Lethbridge Polytechnic, located in southern Alberta, Canada. The field is equipped with a five-span center pivot irrigation system. Before the application of the scheduler, field data collected from May 16, 2022, to August 31, 2022, were used to estimate both soil moisture and soil hydraulic parameters using the extended Kalman filtering approach. Final moisture estimates were used to initialize the scheduler, while hydraulic parameters were used to calibrate the Richards' equation describing soil moisture dynamics.

5.2.1 Simulation Settings

5.2.1.1 Data Generation

The data generation process involved solving the scheduler in a range of weather conditions. The weather inputs included forecasts of reference evapotranspiration, precipitation, and daily average temperature. Temperature forecasts, together with the empirical equations reported in [46], were used to compute crop coefficients for wheat, the crop cultivated in the study area. Historical weather data for the growing seasons 2009 to 2023 were obtained from the Alberta Weather Station database [47]. For each year, data covering the wheat growing period (5 May to 4 September) were extracted. For each growing season, classical GBD was applied to determine irrigation schedules over the specified period, using a scheduling horizon of 14 days. This implementation followed a receding horizon approach, where the calibrated Richards’ equation was used to simulate field conditions. The specific formulations of the master problem and the subproblems solved during GBD are provided in Section 1.2 of the Supplementary Material, and the relevant parameters of $\mathcal{I}(\theta)$ used during data generation are listed in Table 1 of the Supplementary Material. The master problem was solved using Bonmin 1.8.9 [48], while the subproblem was solved using IPOPT 3.12.6 [42].

5.2.1.2 Agent Training

The dataset from Section 5.2.1.1 was used to train the agent in the IL stage. For this example, the network architecture and hyperparameters listed in Table 2 of the Supplementary Material were adopted, the only change being the output layer. Given the 14-day scheduling horizon of the scheduler, the output layer consisted of 14 sigmoid units. In the RL stage, the adopted critic architecture, the reward design parameters, and the hyperparameters of RL training are listed in Table 3 of the Supplementary Material. RL training incorporated historical weather data from two nearby locations (Edmonton and Three Hills Weather Stations) and was carried out over 15,000 episodes. The agent was trained on an Intel® Core™ i7-14700 processor, where the training in the IL stage took 20 minutes while the training in the RL stage took 72 hours.

5.2.1.3 Performance Evaluation

The proposed approach was used to generate wheat irrigation schedules during the 2024 growing season. The confidence thresholds were set at $\delta_1 = 0.10$ and $\delta_2 = 0.90$ to balance computational efficiency with assignment certainty. For comparison, the classical GBD was also applied. A

receding-horizon implementation was adopted, in which the calibrated Richards’ equation was used to represent the actual field dynamics. The simulation used observed 2024 weather data for actual field conditions, while noisy versions of weather inputs were used in scheduler evaluations.

Performance was compared in: (i) predicted crop yield, calculated using the method in [46]; (ii) total prescribed irrigation; and (iii) efficiency of irrigation water use (IWUE), which is defined as predicted yield divided by total prescribed irrigation. Computational performance was analyzed over the 123 daily evaluations conducted from May 5 to September 4, 2024. The assessment included the daily master problem evaluation time, the number of GBD iterations, the daily subproblem evaluation time, the daily scheduler evaluation time, and the total simulation time for the entire growing season.

5.2.2 Results and Discussion

5.2.2.1 Computational Performance

Table 3 summarizes the computational performance of the proposed approach relative to classical GBD. The relatively high standard deviations in the runtimes reflect the day-to-day variability of the scheduling problem, which arises from differences in weather conditions and soil moisture states throughout the growing season. The proposed method reduced the runtime of the master problem by 38%, the runtime on a daily basis by 23%, and the total runtime on a seasonal basis by 23%. However, for the irrigation scheduling problem, the subproblem was the primary bottleneck, since it embeds soil moisture dynamics modeled with an LSTM surrogate of the Richards’ equation. Although simplified, the surrogate retains sufficient complexity, contributing to the high subproblem runtimes. Importantly, accelerating the master problem also improved the efficiency of the subproblem, resulting in a reduction 21% in the runtime of the subproblem. Two factors explain this effect. First, the proposed approach terminated in slightly fewer iterations on average (10 versus 11 for classical GBD), reducing the total number of subproblem evaluations. Second, even after normalizing by iteration count, subproblem evaluations were faster under the proposed method (97 s per subproblem versus 112 s for classical GBD). This provides direct evidence of the agent’s contribution, as its reward structure explicitly penalizes long subproblem times and encourages assignments that yield more tractable subproblems.

Table 3: Comparison of computational runtimes between the proposed and classical GBD approaches over the entire growing season (5 May to 4 September 2024). Values are reported as mean $\pm$ standard deviation across the 123 daily evaluations.

Metric	Proposed	Classical GBD	Improvement (%)
Master problem runtime (s)	83 $\pm$ 18	133 $\pm$ 26	38
Subproblem runtime (s)	973 $\pm$ 1192	1233 $\pm$ 1461	21
Daily runtime (s)	1056 $\pm$ 1208	1366 $\pm$ 1484	23
GBD iterations (count)	10 $\pm$ 12	11 $\pm$ 12	9
Total evaluation time (min)	2165	2800	23

Table 4: Irrigation performance comparison for the 2024 growing season.

Metric	Proposed	Classical GBD	Improvement (%)
Total irrigation (m)	0.35	0.35	–
Predicted yield (kg/m ²)	0.84	0.80	5
IWUE (kg/m ³)	2.41	2.30	5

5.2.2.2 Irrigation Performance

Figure 5 presents the prescribed irrigation schedules and soil moisture trajectories for the 2024 growing season under both the classical GBD and the proposed approach. In both cases, soil moisture was maintained successfully within the target zone, and the total prescribed seasonal irrigation was identical (Table 4). However, Figures 5(a) and (c) show differences in the timing of irrigation events. Except for the first three events, which occur on the same days in both methods, subsequent events are scheduled differently. This discrepancy is expected and is due to the hybrid nature adopted during the agent deployment. In some MPC evaluations, the agent fixes a subset of binary irrigation decisions and the solver then optimizes the remaining variables. This changes the decision space available to the solver. Since MPC minimizes cost over the prediction horizon, the solver adapts the unfixed decisions to best meet the objective. Consequently, the daily irrigation schedules from the proposed method can differ from those of the classical GBD, where the solver retains full control over all binary irrigation decisions. Furthermore, Figures 5(b) and (d) show that the proposed approach led to fewer and smaller violations of the upper bound of the target zone. This reduction is associated with a 5% increase in predicted crop yield. Although both approaches applied the same total irrigation amount, the proposed method used this water more efficiently, resulting in a 5% improvement in IWUE.

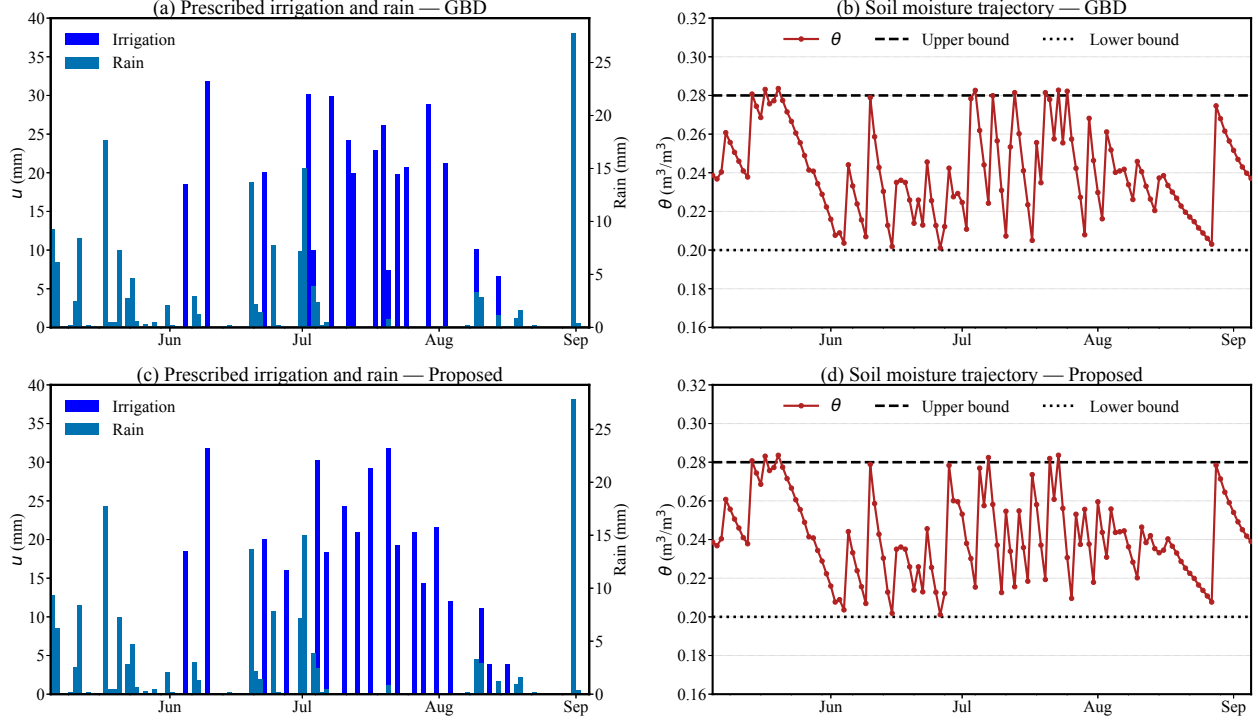


Figure 5: Comparison of prescribed irrigation schedules and corresponding soil moisture trajectories for the 2024 growing season.

5.3 Discussion

One limitation of the proposed approach concerns the termination rule in Algorithm 2, which is based on a surrogate gap between the incumbent UBD and the monotone record of candidate lower-bound values. Since these values are not certified, they may fail to be valid lower bounds of the original MINLP. In such cases, the surrogate-gap test could be satisfied prematurely and Algorithm 2 could return a suboptimal solution. While this behavior was not observed in the case studies considered in this work, the reliability of the surrogate gap can be enhanced by certifying the candidate values. One practical approach is to solve the master problem under a limited computational budget using the accepted assignment from Algorithm 1 as a warm start, and to use the solver-reported best bound in place of the uncertified candidate value.

A second issue is the computational effort for the RL stage. In the first case study, training took about 3 hours, while in the irrigation scheduling case, it required approximately 72 hours on an Intel® Core™ i7-14700 processor. Although these requirements represent a substantial initial investment, training is performed offline. Once trained, the agent can be deployed at a negligible cost. Furthermore, a key practical limitation of the proposed approach is its reliance on training

data, especially in the RL stage, that is representative of the deployment setting. In practice, RL training samples are generated by running GBD on multiple instances, which are obtained by varying the inputs or conditions that define an instance of the original problem (e.g., parameters and exogenous inputs). If the instances encountered during deployment differ substantially from the training distribution, the agent is more likely to propose lower-quality assignments. In such cases, Algorithm 1 is expected to reject a larger fraction of the agent’s actions, which will trigger more frequent fallbacks to the solver and thus reduce the attainable speedups. In extreme cases, poor representativeness can lead to no acceleration, or even worse performance than classical GBD, since the added overhead of Algorithm 1 may outweigh any reduction in solve time.

Regarding the generalization of the proposed approach with respect to problem size, the proposed GNN policy is expected to scale to master problems with a larger number of constraints, provided the same graph construction and feature design used during training are retained. In particular, the policy remains applicable as the master problem grows due to additional constraints, including cut accumulation. However, the current implementation is not designed to generalize directly to instances with a different number of binary decision variables, since the dimension of the action space is tied to the number of binaries used during training. This limitation can be addressed by formulating the action selection of the agent as a node-level task.

Finally, the proposed approach considers the case where the complicating variables of the original MINLP are discrete. Future work could extend it to settings where the complicating variables include both discrete and continuous variables, for example through hybrid policies that combine discrete and continuous actions. Such an extension would broaden the applicability of the proposed framework, and could enable its use within decomposition schemes that leverage GBD to achieve global optimality in nonconvex MINLPs (see, e.g., [49]).

6 Conclusions

This work has demonstrated that graph-based learning can effectively accelerate the master problem in GBD. The proposed framework introduces an agent that processes a bipartite graph of the master problem to predict the values of its binary decision variables. The agent is trained using a two-stage design that integrates IL and RL. In the first stage, a graph neural network with a multi-headed output layer is trained via behavioral cloning. In the second stage, the policy from the first stage is refined within an actor–critic framework using a reward that balances feasibility and

computational efficiency. Integration into GBD is achieved through a verification and coordination step that leverages the agent for high-confidence predictions and falls back to a solver whenever the agent’s assignments fall outside a predefined confidence region, are infeasible, or yield candidate bound values that exceed the incumbent upper bound.

The effectiveness of the approach was illustrated through two case studies. In a generic MINLP, the method accelerated the master problem without compromising the quality of the solution. The results further showed that IL provides a meaningful initial acceleration, while RL builds on this to deliver additional savings without compromising solution quality. In a closed-loop irrigation scheduling problem, formulated as a mixed-integer MPC, the framework accelerated GBD without compromising irrigation performance.

Acknowledgements

The partial financial support of NSF CBET (award number 2313289) is gratefully acknowledged. IM would like to acknowledge financial support from the McKetta Department of Chemical Engineering.

Data Availability

The numerical data for Figures 4 are provided in the Supplementary Material archive under `data/processed/` as `fig4_assignment_frequencies.xlsx`. The time-series data for Figure 5 are included in `data/processed` as `fig6_data.xlsx`. Plotting scripts are available in `code/` as `fig4_plot.py` and `fig6_plot.py`.

Code Availability

The code for all case studies is available at https://github.com/btagyeman/graph_il_rl_for_benders.

References

- [1] P. Daoutidis, J. H. Lee, I. Harjunkski, S. Skogestad, M. Baldea, and C. Georgakis, “Integrating operations and control: A perspective and roadmap for future research,” *Computers*

- & Chemical Engineering*, vol. 115, pp. 179–184, 2018.
- [2] E. N. Pistikopoulos, A. Barbosa-Povoa, J. H. Lee, R. Misener, A. Mitsos, G. V. Reklaitis, V. Venkatasubramanian, F. You, and R. Gani, “Process systems engineering—the generation next?,” *Computers & Chemical Engineering*, vol. 147, p. 107252, 2021.
 - [3] J. F. Benders, “Partitioning procedures for solving mixed-variables programming problems,” *Numerische Mathematik*, vol. 4, no. 1, pp. 238–252, 1962.
 - [4] A. M. Geoffrion, “Generalized Benders decomposition,” *Journal of Optimization Theory and Applications*, vol. 10, no. 4, pp. 237–260, 1972.
 - [5] Y. Luo and M. Ierapetritou, “Design and operation of modular biorefinery supply chain under uncertainty using generalized Benders decomposition,” *AIChE Journal*, vol. 70, no. 8, p. e18458, 2024.
 - [6] C. L. Lara, D. S. Mallapragada, D. J. Papageorgiou, A. Venkatesh, and I. E. Grossmann, “Deterministic electric power infrastructure planning: Mixed-integer programming model and nested decomposition algorithm,” *European Journal of Operational Research*, vol. 271, no. 3, pp. 1037–1054, 2018.
 - [7] I. Mitrai and P. Daoutidis, “Computationally efficient solution of mixed integer model predictive control problems via machine learning aided Benders decomposition,” *Journal of Process Control*, vol. 137, p. 103207, 2024.
 - [8] R. Rahmaniani, T. G. Crainic, M. Gendreau, and W. Rei, “The Benders decomposition algorithm: A literature review,” *European Journal of Operational Research*, vol. 259, no. 3, pp. 801–817, 2017.
 - [9] T. G. Crainic, W. Rei, M. Hewitt, and F. Maggioni, *Partial Benders decomposition strategies for two-stage stochastic integer programs*, vol. 37. CIRRELT, 2016.
 - [10] I. Mitrai and P. Daoutidis, “A multicut generalized Benders decomposition approach for the integration of process operations and dynamic optimization for continuous systems,” *Computers & Chemical Engineering*, vol. 164, p. 107859, 2022.
 - [11] E. Larsen, E. Frejinger, B. Gendron, and A. Lodi, “Fast continuous and integer L-shaped heuristics through supervised learning,” *INFORMS Journal on Computing*, 2023.

- [12] T. L. Magnanti and R. T. Wong, “Accelerating Benders decomposition: Algorithmic enhancement and model selection criteria,” *Operations Research*, vol. 29, no. 3, pp. 464–484, 1981.
- [13] G. R. Raidl, T. Baumhauer, and B. Hu, “Speeding up logic-based Benders’ decomposition by a metaheuristic for a bi-level capacitated vehicle routing problem,” in *International Workshop on Hybrid Metaheuristics*, pp. 183–197, Springer, 2014.
- [14] H. Sohn, B. Tseng, and D. L. Bricker, “A hybrid Benders/genetic algorithm for vehicle routing and scheduling problem,” *International Journal of Industrial Engineering*, vol. 19, no. 1, pp. 33–46, 2012.
- [15] W. Jiang, L. Tang, and S. Xue, “A hybrid algorithm of tabu search and Benders decomposition for multi-product production distribution network design,” in *2009 IEEE International Conference on Automation and Logistics*, pp. 79–84, IEEE, 2009.
- [16] G. R. Raidl, “Decomposition based hybrid metaheuristics,” *European Journal of Operational Research*, vol. 244, no. 1, pp. 66–76, 2015.
- [17] A. M. Geoffrion and G. W. Graves, “Multicommodity distribution system design by Benders decomposition,” *Management Science*, vol. 20, no. 5, pp. 822–844, 1974.
- [18] Z. Li, B. T. Agyeman, I. Mitrai, and P. Daoutidis, “Learning to control inexact Benders decomposition via reinforcement learning,” *arXiv preprint arXiv:2508.06700*, 2025.
- [19] I. Mitrai and P. Daoutidis, “Accelerating process control and optimization via machine learning: A review,” *Reviews in Chemical Engineering*, vol. 41, no. 4, pp. 401–418, 2025.
- [20] P. Gupta, E. B. Khalil, D. Chet  lat, M. Gasse, Y. Bengio, A. Lodi, and M. P. Kumar, “Look-back for learning to branch,” *arXiv preprint arXiv:2206.14987*, 2022.
- [21] M. B. Paulus, G. Zarpellon, A. Krause, L. Charlin, and C. Maddison, “Learning to cut by looking ahead: Cutting plane selection via imitation learning,” in *International conference on machine learning*, pp. 17584–17600, PMLR, 2022.
- [22] H. Jia and S. Shen, “Benders cut classification via support vector machines for solving two-stage stochastic programs,” *INFORMS Journal on Computing*, vol. 3, no. 3, pp. 278–297, 2021.

- [23] R. C. Allen, F. Iseri, C. D. Demirhan, I. Pappas, and E. N. Pistikopoulos, “Improvements for decomposition based methods utilized in the development of multi-scale energy systems,” *Computers & Chemical Engineering*, vol. 170, p. 108135, 2023.
- [24] I. Mitrai and P. Daoutidis, “Taking the human out of decomposition-based optimization via artificial intelligence, Part II: Learning to initialize,” *Computers & Chemical Engineering*, vol. 186, p. 108686, 2024.
- [25] E. Schede, J. Brandt, A. Tornede, M. Wever, V. Bengs, E. Hüllermeier, and K. Tierney, “A survey of methods for automated algorithm configuration,” *Journal of Artificial Intelligence Research*, vol. 75, pp. 425–487, 2022.
- [26] P. Kerschke, H. H. Hoos, F. Neumann, and H. Trautmann, “Automated algorithm selection: Survey and perspectives,” *Evolutionary computation*, vol. 27, no. 1, pp. 3–45, 2019.
- [27] Y. Bengio, A. Lodi, and A. Prouvost, “Machine learning for combinatorial optimization: A methodological tour d’horizon,” *European Journal of Operational Research*, vol. 290, no. 2, pp. 405–421, 2021.
- [28] V. Nair, S. Bartunov, F. Gimeno, I. Von Glehn, P. Lichocki, I. Lobov, B. O’Donoghue, N. Sonnerat, C. Tjandraatmadja, P. Wang, *et al.*, “Solving mixed integer programs using neural networks,” *arXiv preprint arXiv:2012.13349*, 2020.
- [29] P. Daoutidis, W. Tang, and A. Allman, “Decomposition of control and optimization problems by network structure: Concepts, methods, and inspirations from biology,” *AIChE Journal*, vol. 65, no. 10, 2019.
- [30] V.-A. Darvari, S. Hailes, and M. Musolesi, “Graph reinforcement learning for combinatorial optimization: A survey and unifying perspective,” *arXiv preprint arXiv:2404.06492*, 2024.
- [31] C. A. Floudas, *Nonlinear and mixed-integer optimization: Fundamentals and applications*. Oxford University Press, 1995.
- [32] M. Gasse, D. Chételat, N. Ferroni, L. Charlin, and A. Lodi, “Exact combinatorial optimization with graph convolutional neural networks,” *Advances in Neural Information Processing Systems*, vol. 32, 2019.

- [33] A. Allman, W. Tang, and P. Daoutidis, “Decode: A community-based algorithm for generating high-quality decompositions of optimization problems,” *Optimization and Engineering*, vol. 20, no. 4, pp. 1067–1084, 2019.
- [34] W. Tang, A. Allman, D. B. Pourkargar, and P. Daoutidis, “Optimal decomposition for distributed optimization in nonlinear model predictive control through community detection,” *Computers & Chemical Engineering*, vol. 111, pp. 43–54, 2018.
- [35] T. Kipf, “Semi-supervised classification with graph convolutional networks,” *arXiv preprint arXiv:1609.02907*, 2016.
- [36] M. Simonovsky and N. Komodakis, “Dynamic edge-conditioned filters in convolutional neural networks on graphs,” in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pp. 3693–3702, 2017.
- [37] R. S. Sutton, A. G. Barto, *et al.*, *Reinforcement learning: An introduction*, vol. 1. MIT press Cambridge, 1998.
- [38] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, “Proximal policy optimization algorithms,” *arXiv preprint arXiv:1707.06347*, 2017.
- [39] M. Bain and C. Sammut, “A framework for behavioural cloning,” in *Machine intelligence 15*, pp. 103–129, 1995.
- [40] C. A. Poojari and J. E. Beasley, “Improving Benders decomposition using a genetic algorithm,” *European Journal of Operational Research*, vol. 199, no. 1, pp. 89–97, 2009.
- [41] Gurobi Optimization, LLC, “Gurobi Optimizer Reference Manual,” 2024.
- [42] A. Wächter and L. T. Biegler, “On the implementation of an interior-point filter line-search algorithm for large-scale nonlinear programming,” *Mathematical Programming*, vol. 106, no. 1, pp. 25–57, 2006.
- [43] D. Grattarola and C. Alippi, “Graph neural networks in tensorflow and Keras with Spektral,” *Journal of Machine Learning Research*, vol. 21, no. 77, pp. 1–2, 2020.
- [44] F. Chollet *et al.*, “Keras.” <https://github.com/fchollet/keras>, 2015.

- [45] M. Fischetti, F. Glover, and A. Lodi, “The feasibility pump,” *Mathematical Programming*, vol. 104, no. 1, pp. 91–104, 2005.
- [46] B. T. Agyeman, M. Naouri, W. M. Appels, J. Liu, and S. L. Shah, “Learning-based multi-agent MPC for irrigation scheduling,” *Control Engineering Practice*, vol. 147, p. 105908, 2024.
- [47] “Alberta Climate Information Service (ACIS) – Weather Data Viewer.” <https://acis.alberta.ca/weather-data-viewer.jsp>. Accessed: 10 May 2025.
- [48] P. Bonami, L. T. Biegler, A. R. Conn, G. Cornuéjols, I. E. Grossmann, C. D. Laird, J. Lee, A. Lodi, F. Margot, N. Sawaya, and A. Wächter, “An algorithmic framework for convex mixed integer nonlinear programs,” *Discrete Optimization*, vol. 5, no. 2, pp. 186–204, 2008.
- [49] C. Floudas, A. Aggarwal, and A. Ciric, “Global optimum search for nonconvex nlp and minlp problems,” *Computers & Chemical Engineering*, vol. 13, no. 10, pp. 1117–1132, 1989.